

Hands-On-Learning: Designing a Secure Cloud Architecture with Compute, Network, and Storage Controls

Scenario

You are a cloud security engineer tasked with designing a **secure environment** for a new internal application running on AWS. The application will use:

Compute: EC2 instances

Networking: VPC with Security Groups

Storage: S3 buckets and RDS database

Your leadership team wants an **architecture design and policy documentation** that ensures the environment is hardened against common threats (unpatched AMIs, open SSH, public buckets, unencrypted databases).

By completing this lab, you will create a **secure architecture diagram** and a **policy artifact** that demonstrates compliance with best practices.

Instructions for Learners

1 Preparation

Tools you may use:

- AWS Console (sandbox account)

- Diagramming tool (Draw.io, Lucidchart, Miro, or PowerPoint)

Review these AWS services:

EC2 AMIs (hardened images, patching)

Security Groups (ingress/egress rules)

S3 Block Public Access

RDS encryption with KMS

2 Define the Core Components

EC2: Select an official, vendor-provided AMI (e.g., Amazon Linux 2 or Ubuntu LTS).

Networking: Place EC2 instances in private subnets, only accessible via:

- A bastion host, or

- AWS Systems Manager Session Manager.

S3: Buckets default to Block Public Access.

RDS: Encrypt with KMS; disable public accessibility.

Create a draft diagram showing:

- A VPC with subnets

- An EC2 instance for the app server

- An RDS instance

- An S3 bucket for storage

3 Harden Security Controls

AMI Provenance: Document how you'll track AMI IDs in IaC (Terraform/CloudFormation).

Patching: Set up **SSM Patch Manager** with a defined patch baseline.

Firewall Rules:

- Ingress: allow 80/443 from 0.0.0.0/0 to the load balancer only.

- SSH restricted to admin IP (/32) or Session Manager.

- Egress: restrict to HTTPS (443) for OS updates.

S3: Turn on **Block Public Access** and enable bucket versioning.

RDS: Enforce **encryption at rest** and limit IAM roles that can decrypt snapshots.

Update your diagram to reflect these guardrails.

4 Document Guardrails

In a simple table or checklist, define at least **3 AWS Config rules** to enforce posture:

- required-tags → ensure Owner/DataClass tags exist.

- s3-bucket-public-read-prohibited → block public exposure.

- rds-storage-encrypted → enforce encryption.

5 Peer Review

Exchange diagrams with peers.

Ask them to “attack” your design:

- Could an attacker brute force SSH?

- Could data leak from an S3 bucket?

- Are there compliance blind spots?

Adjust your design based on feedback.

Reflection

Which security controls were easiest to implement?

Which trade-offs did you face between **usability** (e.g., open access) and **security** (restrictive rules)?

How would you adapt your design for **multi-region resilience**?

Final Outcome

By completing this lab, you have:

Designed a secure **cloud reference architecture** (compute, network, storage).

Documented **policy guardrails** (Config rules, tagging, encryption).

Produced peer-reviewed artifacts (diagram + policy table) for real-world use.

Next Steps

Implement your design in a sandbox AWS account with IaC (Terraform/CloudFormation).

Expand the architecture to include logging (CloudTrail + GuardDuty).

Share your artifact with classmates or team as part of a **Cloud Security Playbook**.

Instructions for Documenting and Sharing The Project for Peer Review

Document the Project



Use word processing software to create your report.

Ensure all sections of the project document structure are completed.

Proofread and edit your report for clarity and accuracy.

Reflection

Which responsibilities surprised you most (things you thought the provider handled but are actually customer tasks)?

How would this matrix change for a multi-cloud environment?

How could this matrix reduce risk in real-world onboarding and training?

Final Outcome

Created a **Shared Responsibility Matrix** for IaaS, PaaS, and SaaS.

Produced a peer-reviewed artifact suitable for training and executive communication.

Built a reusable tool to guide future cloud migrations and compliance efforts.

Next Steps

Expand the matrix to include **specific services** (EC2, RDS, S3, Google Workspace, etc.).

Layer in **framework mapping** (e.g., CIS, ISO, NIST) to show compliance overlap.

Use this matrix as a foundation for a **Cloud Security Onboarding Guide** in your organization.



Share the Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.



Peer Review

Review the projects submitted by your peers.

Provide constructive feedback and comments on their work.